Dos flujos para llegar al modelo físico

Sin definir el gestor y conociendo el gestor, Forward Engineering y Reverse Engineering

Semana 11 - Sesión 21

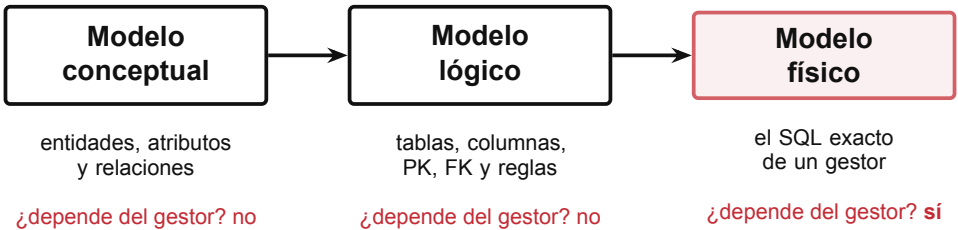
Rel Guzman

Universidad Tecnológica del Perú

Logro de la sesión

Al finalizar, podrás elegir entre dos **flujos de modelado**: diseñar sin haber definido el gestor, con **DBML** u otra herramienta, o diseñar sabiendo el gestor desde el inicio, directamente en **SQL**. También explicarás qué hacen **Forward Engineering** y **Reverse Engineering**, y llevarás un mismo modelo lógico a **tres gestores**.

Revisión de la sesión anterior: tres niveles de detalle

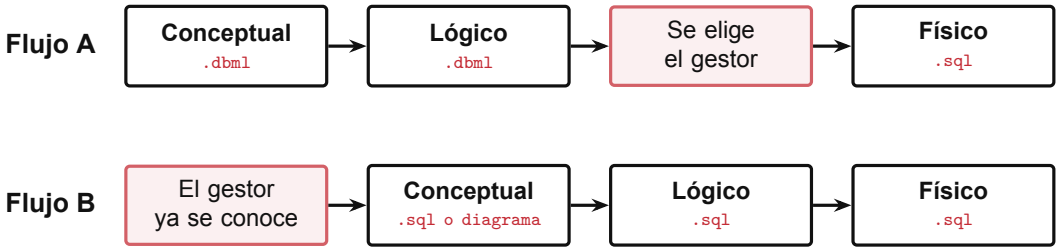


La pregunta de hoy

Solo el físico depende del gestor. Entonces, ¿**en qué momento** conviene elegirlo: al final del diseño o desde el principio?

Seguimos con el caso de la **peluquería de mascotas**: carpeta `ejemplos/` de esta sesión, con **un proyecto por flujo**.

Dos flujos: ¿cuándo se elige el gestor?



Flujo A Todavía **no se sabe** dónde vivirá la base. Se diseña con una herramienta **neutral** y el gestor se elige al final.

Flujo B El gestor **ya está decidido**: por ejemplo, la empresa ya usa **MySQL**. Se diseña **directamente en SQL**.

En los dos flujos existen los **tres modelos**. Lo que cambia es **en qué se escriben** y **cuándo** aparece el gestor.

Los ejemplos: un proyecto por flujo

Los dos proyectos resuelven **el mismo caso** y tienen la misma carpeta `modelado/`. Lo que cambia es **en qué** se escribe cada modelo.

Flujo A: sin gestor

```
ejemplos/
   proyecto_mascotas_flujo_sin_gestor/
      modelado/
         1_contexto.md
         2_modelo_conceptual.dbml
         3_modelo_logico.dbml
         4_modelo_fisico_mysql.sql
         4_modelo_fisico_sqlserver.sql
         4_modelo_fisico_sqlite.sql
```

Flujo B: con gestor

```
ejemplos/
   proyecto_mascotas_flujo_con_gestor/
      modelado/
         1_contexto.md
         2_modelo_conceptual_comentarios.sql
         2_modelo_conceptual_tablas.sql
         2_modelo_conceptual.mwb
         3_modelo_logico.sql
         4_modelo_fisico.sql  -- solo MySQL
```

- Los números indican el **orden**: contexto, conceptual, lógico y físico.
- Flujo A: un lógico neutral y **un físico por gestor**, porque el gestor aún no se decide.
- Flujo B: el gestor ya es MySQL, así que hay **un solo físico**.

Flujo A

Flujo A: el gestor todavía no está definido

¿Cuándo pasa?

- El cliente aún no decide si usará MySQL, SQL Server u otro gestor.
- El diseño se revisa con personas que **no programan**.
- El mismo diseño podría terminar en **más de un** gestor.

Cómo se trabaja

- Conceptual y lógico en una herramienta **neutral**.
- Al final se elige el gestor y se **genera** el SQL del físico.
- Proyecto: `proyecto_mascotas_flujo_sin_gestor/`.

DBML: Database Markup Language

Un lenguaje de **texto** para describir tablas y relaciones. **dbdiagram.io** lo dibuja como diagrama y lo exporta a SQL de MySQL, PostgreSQL, SQL Server u Oracle. Otras opciones: **Mermaid**, **ERDPlus** o **draw.io**.

Flujo A: así se ve el conceptual en DBML

```
Table cliente {
   id integer [pk]
   nombre varchar(255)
   telefono varchar(255)
}

Table mascota {
   id integer [pk]
   nombre varchar(255)
   especie varchar(255)
   raza varchar(255)
   cliente_id integer [not null]
}

// @relation-meta
// sourceLabel="es dueño de"
// targetLabel="pertenece a"
Ref: cliente.id < mascota.cliente_id
```

- Cada **entidad** es un `Table` con sus **atributos**.
- La **relación** es un `Ref`. El < se lee «uno a muchos».
- Los **verbos** de la relación quedan en comentarios.
- `varchar(255)` es solo **de relleno**: DBML exige un tipo, pero aquí todavía no se decide.

Archivo: `2_modelo_conceptual.dbml`

El mismo conceptual en Mermaid

```
erDiagram
    CLIENTE  ||--|{ MASCOTA : "es dueño de"
    MASCOTA  ||--|{ CITA    : "tiene"
    SERVICIO ||--o{ CITA    : "se presta en"
    EMPLEADO ||--o{ CITA    : "atiende"
    CITA     ||--o| PAGO    : "se paga con"
```

- Solo **entidades**, **relaciones** y sus **verbos**: no pide claves ni tipos.
- Los símbolos son la **pata de gallo**: `||` es «uno», `|{` es «uno o muchos», `o{` es «cero o muchos» y `o|` es «cero o uno».

Se lee: `CLIENTE ||--|{ MASCOTA` → «un cliente es dueño de **una o muchas** mascotas; cada mascota es de **un** cliente».

En palabras simples

Mermaid sirve para **conversar** el conceptual. Para el lógico conviene DBML: allí ya se escriben PK, FK y reglas que luego se convierten en SQL.

Flujo A: así se ve el lógico en DBML

```
Table cita {
   id integer [pk, increment]
   fecha_hora datetime [not null]
   estado varchar(20) [not null, default: 'programada',
      check: `estado IN ('programada',
                'completada', 'cancelada')`]
   mascota_id  integer [not null]
   servicio_id integer [not null]
   empleado_id integer [not null]
   indexes { (empleado_id, fecha_hora) [unique] }
}
Ref: mascota.id  < cita.mascota_id
Ref: servicio.id < cita.servicio_id
Ref: empleado.id < cita.empleado_id
```

- Se agregan **tipos**, **PK**, obligatorios, valores por defecto y reglas.
- Cada `Ref` se convertirá en una **FK**.
- **Sin nada propio de un gestor**: tipos comunes y reglas que cualquiera entiende.

Archivo: `3_modelo_logico.dbml`

Ojo con esto

El lógico de la sesión anterior usaba `typeof()` y `COLLATE NOCASE`: eso es **de SQLite**. En el flujo A se deja para el físico.

Flujo A: recién al final se elige el gestor

Desde el mismo `3_modelo_logico.dbml` se genera el SQL de cada gestor:

MySQL

```
CREATE TABLE `cliente` (
   `id` integer PRIMARY KEY AUTO_INCREMENT,
   `nombre` varchar(100) NOT NULL,
   `telefono` varchar(20) UNIQUE
);
```

SQL Server

```
CREATE TABLE [cliente] (
   [id] integer PRIMARY KEY IDENTITY(1, 1),
   [nombre] varchar(100) NOT NULL,
   [telefono] varchar(20) UNIQUE
)
GO
```

- Un **mismo lógico**, dos físicos: cambian la numeración del `id`, las comillas y los separadores.
- Lo generado se **revisa** y se completa con los **datos iniciales**: así quedan `4_modelo_fisico_mysql.sql` y `4_modelo_fisico_sqlserver.sql`.

Flujo A: el SQL generado se revisa

El generador traduce la sintaxis, pero **no conoce el negocio**. Antes de ejecutar el físico, se revisa:

- **Nulos en columnas únicas.** En SQL Server, `telefono UNIQUE` acepta **un solo** `NULL`: el segundo cliente sin teléfono sería rechazado. Por eso ese físico usa un **índice filtrado** (`WHERE telefono IS NOT NULL`).
- **Mayúsculas.** Si `'Corte'` y `'corte'` deben ser el mismo servicio, depende de la **intercalación** (*collation*) que use cada gestor.
- **Gestores no incluidos.** `dbml2sql` no genera SQL de **SQLite**: `4_modelo_fisico_sqlite.sql` se escribe a mano.

Idea clave del flujo A

Conceptual y lógico **se escriben una sola vez**. Por cada gestor se genera y se revisa un físico distinto.

Flujo B

Flujo B: el gestor se conoce desde el inicio

¿Cuándo pasa?

- La empresa ya trabaja con **un** gestor.
- La peluquería contrató un hosting que trae **MySQL**.
- El equipo prefiere escribir **SQL** desde el principio.

Cómo se trabaja

- Los tres modelos se escriben en archivos `.sql`.
- El lógico ya se puede **ejecutar** y probar en el gestor.
- Proyecto: `proyecto_mascotas_flujo_con_gestor/`.

SQL como lenguaje de modelado

El diseño vive en el mismo lenguaje que crea la base. **MySQL Workbench** lo muestra como diagrama EER, ya atado a MySQL.

Flujo B: tres formas de escribir el conceptual

El conceptual dice **qué se guarda** y **cómo se relaciona**. En el flujo B se puede escribir de tres maneras:

	Cómo se ve	Archivo
Alternativa 1	solo comentarios en SQL	`2_modelo_conceptual_comentarios.sql`
Alternativa 2	tablas y columnas, sin reglas	`2_modelo_conceptual_tablas.sql`
Alternativa 3	diagrama EER en MySQL Workbench, sin tipos	`2_modelo_conceptual.mwb`

Lo que tienen en común

Ninguna define **PK**, **FK** ni reglas. Eso se decide en el **lógico**. Se elige la que el equipo entienda mejor.

Conceptual, alternativa 1: solo comentarios

```
-- ENTIDADES Y ATRIBUTOS
--   cliente  : nombre, teléfono
--   mascota  : nombre, especie, raza
--   cita     : fecha y hora, estado
--   …
-- RELACIONES
--   cliente  1 - N  mascota  es dueño de
--   mascota  1 - N  cita     tiene
--   empleado 1 - N  cita     atiende
--   cita     1 - 1  pago     se paga con
-- REGLAS DEL NEGOCIO
--   Un empleado no atiende dos citas
--   a la misma hora.
```

- Son solo **comentarios**: el gestor no ejecuta nada.
- Se lee como una **lista**: entidades, relaciones con su verbo y reglas.
- Lo más rápido para empezar, pero no se ve como diagrama.

Archivo:
`2_modelo_conceptual_comentarios.sql`

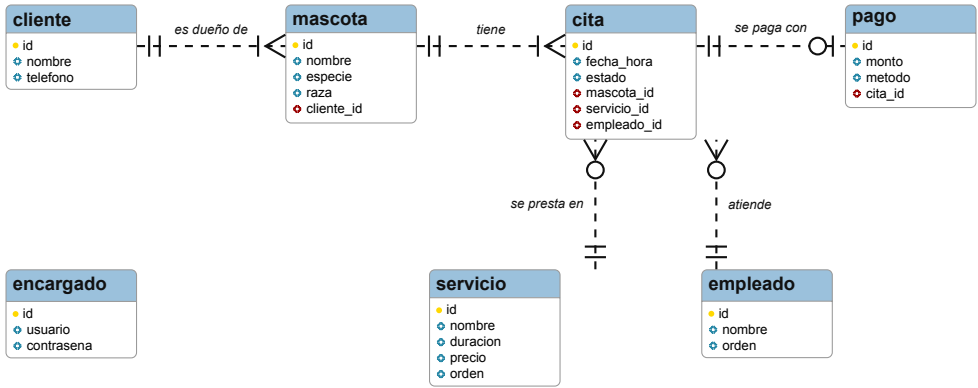
Conceptual, alternativa 2: tablas y columnas, sin reglas

```sql
CREATE TABLE cliente (
   id INT,
   nombre VARCHAR(100),
   telefono VARCHAR(100)
);
CREATE TABLE mascota (
   id INT,
   nombre VARCHAR(100),
   especie VARCHAR(100),
   raza VARCHAR(100),
   cliente_id INT  -- es dueño de
);
-- … y lo mismo para las demás
```

- Solo **tablas**, **columnas** y dos tipos provisionales: `INT` y `VARCHAR`.
- Sin `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL` ni `UNIQUE`.
- `cliente_id` solo **anuncia** la relación; todavía no la exige.
- Ya se puede ejecutar en MySQL para ver las tablas.

Archivo: `2_modelo_conceptual_tablas.sql`

Conceptual, alternativa 3: diagrama en Workbench



- Diagrama **EER** de **MySQL Workbench**, hecho a partir de `2_modelo_conceptual_tablas.sql`.
- Tipos ocultos: *Model Options* → desmarcar *Show Column Types*. Cada línea lleva su **verbo**.

Archivo: `2_modelo_conceptual.mwb`

Flujo B: así se ve el lógico en SQL

```sql
CREATE TABLE servicio (
   id INT PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL UNIQUE,
   duracion INT NOT NULL CHECK (duracion > 0),
   precio DECIMAL(8,2) NOT NULL CHECK (precio > 0),
   orden INT NOT NULL
);

CREATE TABLE cita (
   id INT PRIMARY KEY,
   fecha_hora DATETIME NOT NULL,
   estado VARCHAR(20) NOT NULL DEFAULT 'programada'
      CHECK (estado IN ('programada', 'completada',
                'cancelada')),
   …
   FOREIGN KEY (servicio_id) REFERENCES servicio(id),
   UNIQUE (empleado_id, fecha_hora)
);
```

- Tablas, columnas, **PK**, **FK** y reglas, escritas como `CREATE TABLE`.
- Como el gestor se conoce, ya se usan sus tipos: `INT`, `VARCHAR`, `DECIMAL`, `DATETIME`.
- Se puede **ejecutar** en MySQL para probarlo.

Archivo: `3_modelo_logico.sql`

Flujo B: el físico completa al lógico

```sql
CREATE DATABASE IF NOT EXISTS peluqueria
   CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS servicio (
   id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL UNIQUE,
   duracion SMALLINT UNSIGNED NOT NULL CHECK (duracion > 0),
   precio DECIMAL(8,2) NOT NULL CHECK (precio > 0),
   orden INT UNSIGNED NOT NULL
) ENGINE = InnoDB;

-- en cita, el estado pasa a ser un ENUM
estado ENUM('programada', 'completada', 'cancelada')
   NOT NULL DEFAULT 'programada'
```

- Es el **mismo diseño**, con lo que **solo MySQL** tiene.
- `AUTO_INCREMENT`: MySQL numera el `id`.
- `InnoDB`: el motor que respeta las FK.
- `utf8mb4`: tildes, ñ y emojis.
- `ENUM`: solo acepta los valores de la lista.

Archivo: `4_modelo_fisico.sql`

Flujo B: lo que el físico agrega, probado en MySQL

Se intenta guardar	Solo con el lógico	Con el físico
un cliente **sin indicar** su `id`	error: `Field 'id' doesn't have a default value`	se guarda con `id = 1`
una cita de la mascota `99`, que no existe, si el servidor crea las tablas con MyISAM	se guarda	rechazada: `a foreign key constraint fails`

- MySQL 8 ya usa InnoDB y `utf8mb4` **por defecto**. El físico los escribe para **no depender** de cómo esté configurado el servidor.

Idea clave del flujo B

Los tres modelos están en SQL. El físico es el lógico **más los detalles del gestor**.

Los dos flujos, lado a lado

	Flujo A: sin gestor	Flujo B: con gestor
Se elige el gestor	al final, antes del físico	antes de empezar
Conceptual	DBML o Mermaid	comentarios, tablas sin reglas o diagrama
Lógico	DBML con tipos comunes	`CREATE TABLE` del gestor
Físico	SQL **generado** y revisado	el lógico **completado** a mano
A favor	un diseño, varios gestores	se prueba desde el inicio
En contra	el SQL generado puede fallar	cambiar de gestor obliga a reescribir

Decide y explica

Una municipalidad aún no compra el servidor y quiere aprobar el diseño. ¿Qué flujo usarías? ¿Y para agregar una tabla a la aplicación de la peluquería?

Forward y Reverse

Forward Engineering: del modelo a la base de datos

Ingeniería directa

En español: **ingeniería directa** (también «ingeniería hacia adelante»). Parte de un **modelo** y genera el SQL que crea la estructura de la base. Puede exportar un script o ejecutarlo en un servidor.

1. Definir tablas, tipos, claves y restricciones en el modelo.
2. Generar y revisar las instrucciones `CREATE TABLE` y los demás objetos.
3. Ejecutar el script para crear la estructura de la base de datos.

Cómo se ve en la realidad

En el flujo A, `dbml2sql` o el botón *Export* de dbdiagram.io hacen ingeniería directa. En MySQL Workbench, la opción *Forward Engineer* genera el script desde su diagrama y lo ejecuta en MySQL.

Crea las tablas **vacías**: no llena los datos de clientes o citas.

Reverse Engineering: de la base de datos al modelo

Ingeniería inversa

En español: **ingeniería inversa**. Parte de una **base de datos existente** o de su script de creación y reconstruye un modelo de su estructura.

1. Leer las tablas, columnas, claves y restricciones existentes.
2. Importarlas en una herramienta de modelado.
3. Revisar el modelo obtenido para documentar o modificar la base.

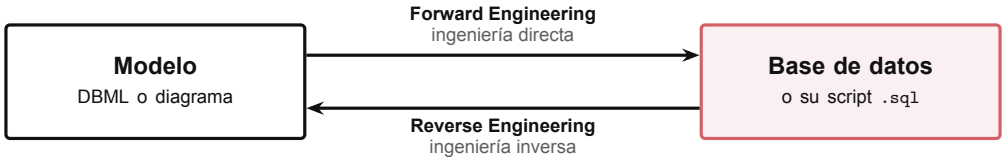
Cómo se ve en la realidad

`sql2dbml` y el *Import* de dbdiagram.io convierten un script SQL en DBML. MySQL Workbench tiene *Reverse Engineer*. En SQLite, `.schema` o DB Browser muestran los `CREATE TABLE` de un archivo `.db`.

Ojo con esto

Recupera solo lo que está **declarado**. Al pasar el SQL de MySQL por `sql2dbml`, la relación 1 a 1 entre cita y pago vuelve como 1 a muchos, y los verbos desaparecen.

Directa e inversa: el mismo camino en dos sentidos



	Forward	Reverse
Parte de	el modelo	la base o su script
Produce	el script `CREATE TABLE`	el modelo o diagrama
Sirve para	crear la base	documentar una base sin modelo
En la práctica	generar el físico con `dbml2sql`	volver al DBML con `sql2dbml`

Forward y Reverse en MySQL Workbench

En Workbench, el modelo es un **diagrama EER** (entidad-relación extendido): las tablas, columnas y relaciones dibujadas.

Proceso	Dirección	Resultado
Forward Engineering	Modelo EER → SQL	Genera un script `.sql`
Forward Engineering	Modelo EER → BD	Crea tablas y relaciones en MySQL
Reverse Engineering	BD → Modelo EER	Genera el diagrama de una BD existente
Reverse Engineering	SQL → Modelo EER	Reconstruye el modelo desde un script

En palabras simples

Cada proceso tiene dos variantes: trabajar con el **archivo** `.sql` o directamente con la **base de datos** en el servidor.

Práctica

Práctica Un mismo lógico, tres gestores

Partimos del lógico del flujo A:
`proyecto_mascotas_flujo_sin_gestor/modelado/3_modelo_logico.dbml`.

SQL Server En **SSMS**, ejecuta `4_modelo_fisico_sqlserver.sql` y revisa las tablas creadas.

MySQL En **MySQL Workbench**, ejecuta `4_modelo_fisico_mysql.sql`. Luego usa *Reverse Engineer* para ver el diagrama EER.

SQLite Escribe el físico **a mano** a partir del lógico y ábrelo en **DB Browser for SQLite**.

Decide y explica

¿Qué cambió entre los tres scripts y qué se mantuvo igual?

¿Por qué esto es pasar del lógico al físico?

Lo que no cambió en los tres

- Las mismas tablas y columnas.
- Las mismas PK, FK y reglas.
- Es decir, **el modelo lógico**.

Lo que sí cambió

- Cómo se numera el `id`.
- Qué tipo guarda fechas y dinero.
- Cómo trata los `NULL` un `UNIQUE`, o si revisa las FK.
- Es decir, **el modelo físico**.

Del lógico al físico

Pasar del lógico al físico es **tomar las decisiones de un gestor concreto** sin cambiar el diseño. Cada diferencia que encontraste es una de esas decisiones.

Por eso el flujo A genera un físico distinto por gestor, y en el flujo B el físico se escribe ya para uno solo.

Preguntas



Cierre: lo de hoy en una tabla

	En una frase
Flujo A: sin gestor	Conceptual y lógico neutrales; el físico se genera al final.
Flujo B: con gestor	Los tres modelos en SQL; el físico completa al lógico.
DBML y Mermaid	Texto que se dibuja como diagrama del conceptual y el lógico.
Forward Engineering	Ingeniería directa: del modelo al script que crea la base.
Reverse Engineering	Ingeniería inversa: de la base existente al modelo.
Del lógico al físico	El mismo diseño, con las decisiones de un gestor concreto.

Elegir el gestor antes o después **no cambia los tres modelos**: cambia **en qué** se escriben.

Referencias

- Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración*. Cengage Learning.
- Holistics. *DBML: Database Markup Language* y *DBML CLI*.
- Mermaid. *Entity Relationship Diagrams*.
- Oracle. MySQL Workbench Manual: *Forward Engineering* y *Reverse Engineering*.
- Microsoft Learn. *Create unique constraints*.
- Oracle. MySQL Reference Manual: *Using AUTO_INCREMENT* y *FOREIGN KEY Constraints*.



UTP Universidad Tecnológica del Perú